Transfer Learning and CNNs in Practice

Standing on ImageNet's shoulders

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The 2,000-image reality

The sorting line finally needs its own model: grade each fruit as **export**, **market**, or **juice**. You spent a week collecting and labeling **2,000 photos**.

Predict: you train the [49] ResNet-18 on them, from scratch. What happens?



2,000 of these - and that is all you get.

The 2,000-image reality

The sorting line finally needs its own model: grade each fruit as **export**, **market**, or **juice**. You spent a week collecting and labeling **2,000 photos**.

Predict: you train the [49] ResNet-18 on them, from scratch. What happens?

It **memorizes** the training set and falls apart on new fruit: **11.7 million** parameters vs **2,000** examples - the overfitting story ([06]) at its bluntest.

The fix is not a smaller net. It is to **not start from scratch**.



2,000 of these - and that is all you get.

Outline

Transfer learning

Bigger shoulders

Seeing what the net sees

The economics of small data

Reuse features, not just architectures

Somebody already trained a net on 1.2 million images. Most of what it learned is about *seeing*, not about their classes.

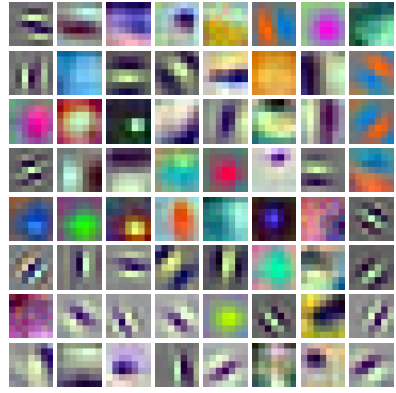
Why transfer works

[48]: conv layers learn a **feature hierarchy** - edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects.

[49]: a pretrained net's conv1 filters *are* the kernel zoo - edge and color detectors nobody designed.

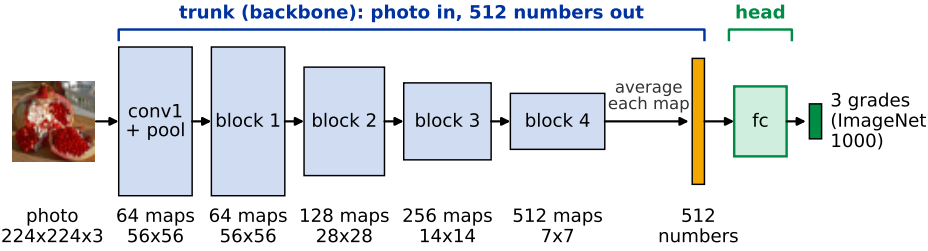
Edges and textures are the **same** for pomegranates, pets, and X-rays. Only the late, task-specific layers need *your* data.

A pretrained network's conv layers are a general-purpose **visual feature extractor**. The hierarchy is the reusable asset.



The [49] payoff figure: learned on ImageNet, useful everywhere.

ResNet-18: a trunk and a head



From block 2 on, each block **halves** the height and width and **doubles** the channels. Inside: two residual units of two 3×3 convs ([49]).

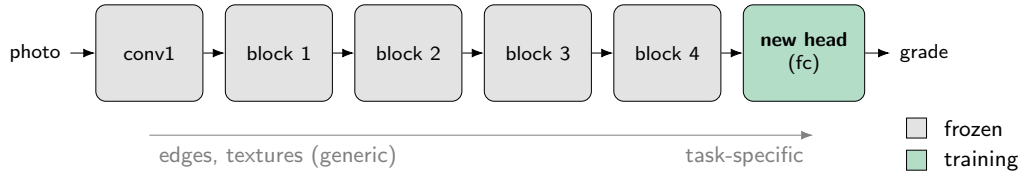
Where 512 comes from: block 4 ends with 512 maps of 7×7; global average pooling ([49]) turns each map into one number. Those **512 numbers** are all the head ever sees.

Why "trunk"? Body words: the **head** sits on top and makes the call; the **trunk** (torso) carries almost all the weights. Keep the trunk, swap the head. Most papers say **backbone** - same part; [51] uses that word.

18 = conv1 + 16 convs in the blocks + fc. PyTorch names: `conv1`, `layer1-layer4`, `avgpool`, `fc`.

Two recipes

(Click to unfreeze.)

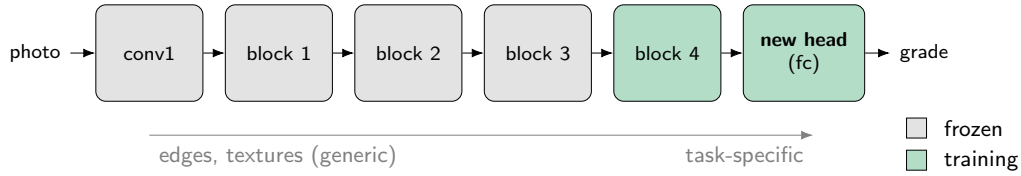


Recipe 1 - feature extraction. Freeze the whole trunk; train only a new head. The trunk is a fixed feature extractor - tiny data suffices.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

Two recipes

(Click to unfreeze.)

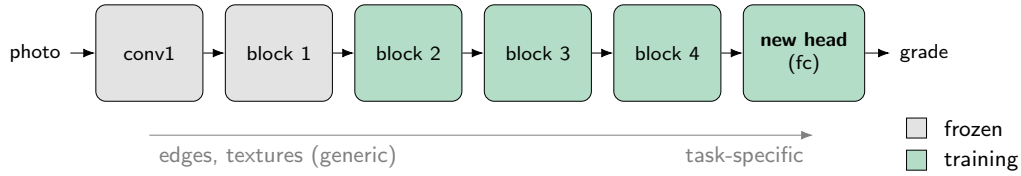


Recipe 2 - fine-tuning. Start by unfreezing the *top* block - the most task-specific features - with a **small** learning rate (LR).

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

Two recipes

(Click to unfreeze.)

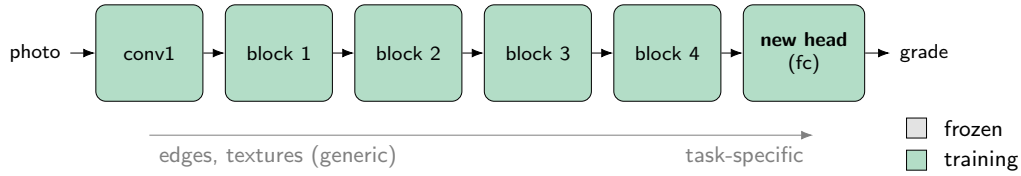


With more data, unfreeze deeper. Early layers stay frozen longest - their features are the most generic.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

Two recipes

(Click to unfreeze.)



With enough data, everything trains - still at a small LR. Better ceiling, more ways to overfit.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

How much does each recipe actually train?

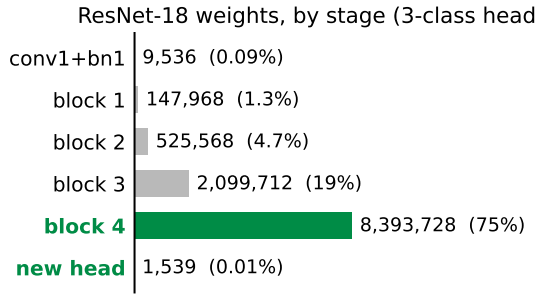
Predict: Recipe 2 starts by unfreezing *one* of the five stages - the top block. What share of the weights is that?

How much does each recipe actually train?

Predict: Recipe 2 starts by unfreezing *one* of the five stages - the top block. What share of the weights is that?

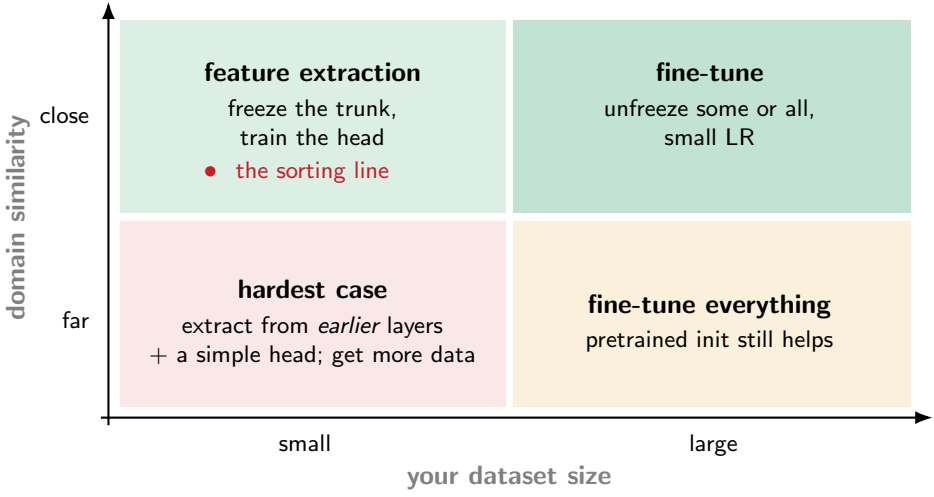
Count them (3-grade head; **green** = what the first unfreeze trains):

- head only: $512 \cdot 3 + 3 = \mathbf{1{,}539}$ - 512 numbers in (block 4's maps, averaged), 3 grades out, plus 3 biases;
- + block 4: **8.4M** - **75%** of the net;
- everything: **11.2M** (11.7M minus ImageNet's 1000-class head).



Why the top is heavy: channels double every stage and a 3×3 conv has $9 \cdot C_{\text{in}} \cdot C_{\text{out}}$ weights - double both, $4\times$ the weights. And Recipe 1 fits **1,539** numbers to **2,000** photos: [11]'s logistic regression on a $2{,}000 \times 512$ table - the cold open's problem, reversed.

Which recipe? Two questions decide



Fine-tuning in practice: you know all of this

- **Small learning rate.** You start *near a good minimum* - a big step jumps away from everything ImageNet taught the net ([44]'s learning-rate intuition).
- **Augmentation on.** Flips, crops, color jitter stretch 2,000 photos further ([43]).
- **Early stopping** on a validation split - the overfitting guardrail ([06], [43]).
- **Freeze first, unfreeze gradually** - the one genuinely new move today. Why it matters is the next slide.

Fine-tuning is the [42]-[45] toolkit applied near a good minimum. Nothing new to learn - just new discipline.

Why freeze first? A random head bends the trunk

Predict: you unfreeze everything from step one, with a freshly initialized head. What do the first few hundred gradient steps do to the pretrained trunk?

Why freeze first? A random head bends the trunk

Predict: you unfreeze everything from step one, with a freshly initialized head. What do the first few hundred gradient steps do to the pretrained trunk?

- The new head starts **random**: its outputs are noise, so the loss is large and so are its gradients.
- Backprop ([42]) carries those large, meaningless gradients **into the trunk**. The pretrained features get bent to suit a head that knows nothing yet.
- **Fix:** train the head alone first (Recipe 1 with a linear head - a **linear probe**), then unfreeze at a small learning rate: **linear probing, then fine-tuning** (**LP-FT**).

Measured (Kumar et al., ICLR 2022, 10 datasets): plain fine-tuning beat the linear probe by 2% on test photos like the training ones, but **lost by 7%** on shifted ones - bent features generalize worse. LP-FT beat plain fine-tuning by 1% and 10%.

One step further: a learning rate per depth, largest for the head (*discriminative fine-tuning*, Howard & Ruder 2018) - generic features move least.

The BatchNorm gotcha

You froze the trunk: `requires_grad = False` on every weight. Accuracy is still bad. Why?

- In `train()` mode BatchNorm (BN) normalizes each batch with **that batch's** mean and variance, and keeps updating **running averages** of them; `eval()` uses the running averages instead ([44] owns the formula).
- Those running averages are **not weights**: `requires_grad = False` does not touch them, so they keep changing while "everything is frozen".
- Small batches from *your* small dataset make the per-batch statistics noisy and drag the running averages away from the ImageNet values the frozen weights expect - accuracy drops **silently**.

How fast? PyTorch updates running $\leftarrow 0.9 \cdot \text{running} + 0.1 \cdot \text{batch}$. Measured: a frozen BN layer in `train()` mode, stored mean 0, fed 10 batches of mean 5 $\rightarrow$ running mean 3.26 - already 65% of the way ($1 - 0.9^{10}$) to your data.

Practical rule: when the trunk is frozen, put its BN layers in **eval mode**. And whenever a "frozen" model underperforms - suspect the BN statistics first.

The canonical snippet

The whole of Recipe 1 - including the BatchNorm rule:

```
from torchvision import models
import torch, torch.nn as nn

model = models.resnet18(weights="IMAGENET1K_V1")  # 11.7M trained weights
for p in model.parameters():
    p.requires_grad = False                       # freeze the trunk
model.fc = nn.Linear(model.fc.in_features, 3)     # new head: 3 grades
model.eval()                                      # BN: keep ImageNet stats
optimizer = torch.optim.SGD(model.fc.parameters(),
                            lr=0.001, momentum=0.9)
# then: the usual loop, minus model.train()
```

The new `fc` is created *after* the freeze, so it trains by default - and it is the only thing the optimizer sees.

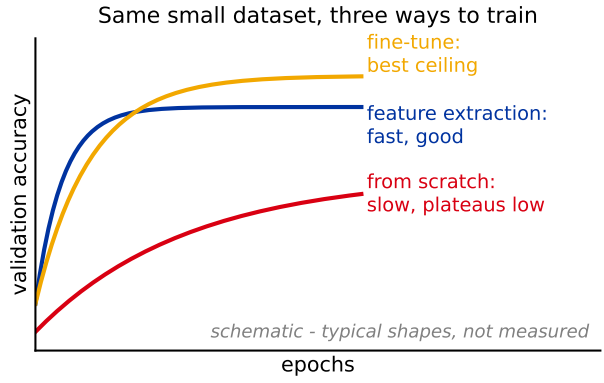
`eval()` **is not "no gradients".** It only switches BatchNorm (and dropout) to inference behaviour; gradients still reach `fc`. Gradients are a separate switch: `requires_grad`.

What transfer buys you

Same small dataset, three ways to train:

- **from scratch**: slow, plateaus low - 11.7M parameters memorize 2,000 photos;
- **feature extraction**: fast and good;
- **fine-tune**: the best ceiling.

Typical shapes, not measurements. Gesture Snake measures two of them on your own photos: from scratch vs a frozen ResNet-18.



When transfer fails - or stops mattering

- **Domain too far.** Spectrograms, medical volumes, radar - ImageNet features stop being the right vocabulary. Unfreeze more, or rethink.
- **Bias mismatch.** ImageNet is object-centric daylight photography; data unlike that fights the pretrained assumptions.
- **Lots of data.** He, Girshick & Dollár (2019) trained COCO detectors from scratch and matched ImageNet-pretrained ones - given longer training. Pretraining bought **speed**, not a higher ceiling. 2,000 photos is nowhere near that regime.

Classic silent bugs - the input contract: the pretrained net expects **its own** preprocessing, and quietly underperforms on anything else.

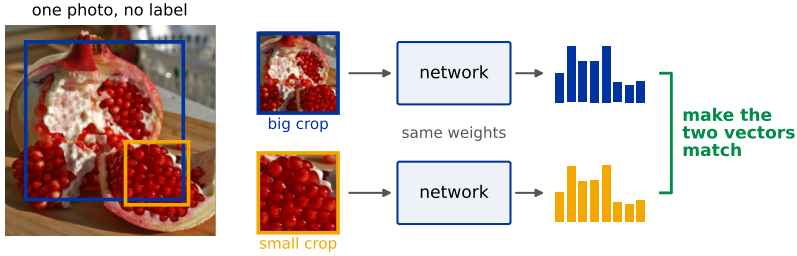
- ImageNet's mean $(0.485, 0.456, 0.406)$ and std $(0.229, 0.224, 0.225)$ - *not* your dataset's;
- `cv2.imread` gives blue-green-red (**BGR**); the net wants RGB;
- simplest: `ResNet18_Weights.IMAGENET1K_V1.transforms()`.

Bigger shoulders, no labels

Everything so far stood on ImageNet: 1.2 million photos, each labeled by a person. The newest trunks learned from 400 million to 1.7 billion photos that nobody labeled by hand.

Shoulders without labels: self-supervised learning

Labels are the expensive part. Can a network learn to see from **photos alone**?



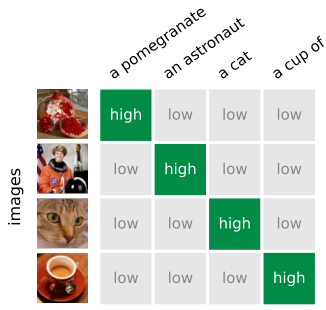
Why it learns content: a wide view and a close-up of the arils look very different pixel by pixel, but show the same thing. Matching vectors are only possible if they encode *what* is there.

The catch: "the same vector for every photo" matches perfectly too. DINO avoids that collapse with a slowly updated "teacher" copy of the network and by centering its outputs.

Meta's **DINOv3** (2025): a 7-billion-parameter vision transformer trained this way on 1.7 billion unlabeled photos, built to be used **frozen** - and distilled into laptop-sized versions, including ConvNeXt CNNs ([49]).

Shoulders with words: CLIP

OpenAI's **CLIP** (2021) learned from **400 million image-caption pairs** from the web - nobody labelled them; the captions came for free.



One batch: 4 images, 4 captions.

Two encoders, one for images, one for text.
Training pushes each image towards its own caption (diagonal) and away from the other 3 captions.

Afterwards: classify a photo by writing the class names as captions.

Zero-shot: embed "a photo of a pomegranate", "a photo of an apple", ..., and pick the caption closest to the photo. With no ImageNet training at all, CLIP matched the original ResNet-50 on ImageNet.

2026: whose shoulders you stand on

Same two recipes - but the trunk is increasingly **not** an ImageNet classifier: it is a self-supervised (DINOv3) or image-text (CLIP) backbone.

- The **linear probe** from the LP-FT slide - freeze the trunk, fit one linear layer (logistic regression, [11]) on its features - is how these backbones are graded: the better the frozen features, the better a plain linear model does on top.
- The Gesture Snake practical does exactly this with a frozen ResNet-18.

The sorting line's first try in 2026: frozen DINOv3 or CLIP features + a linear head. Fine-tune only if that is not enough. Bigger shoulders, and nobody had to label them.

Trust, but verify

The fine-tuned model scores well. Before it grades a single truckload: *what is it actually looking at?*

Grad-CAM: where is the net looking?

Three steps, no new math:

1. Take the **last conv block's feature maps** - 512 coarse 7×7 maps in ResNet-18: the most abstract "where" information left.

the photo (224 x 224)

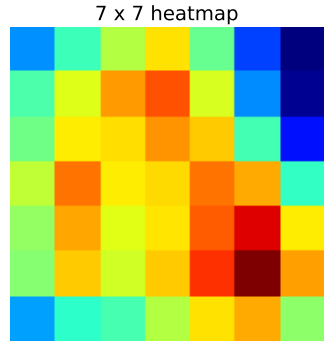


Grad-CAM = Gradient-weighted Class Activation Mapping (Selvaraju et al., ICCV 2017).

Grad-CAM: where is the net looking?

Three steps, no new math:

1. Take the **last conv block's feature maps** - 512 coarse 7×7 maps in ResNet-18: the most abstract "where" information left.
2. **Weight each map by the gradient** of the class score with respect to it - one number per map: how much the score would rise if that map fired harder.



Grad-CAM = Gradient-weighted Class Activation Mapping (Selvaraju et al., ICCV 2017).

Grad-CAM: where is the net looking?

Three steps, no new math:

1. Take the **last conv block's feature maps** - 512 coarse 7×7 maps in ResNet-18: the most abstract "where" information left.
2. **Weight each map by the gradient** of the class score with respect to it - one number per map: how much the score would rise if that map fired harder.
3. **Sum, ReLU, upsample** - a heatmap of the evidence *for* that class.

One forward, one backward, any trained CNN. Next: each step on our photo.

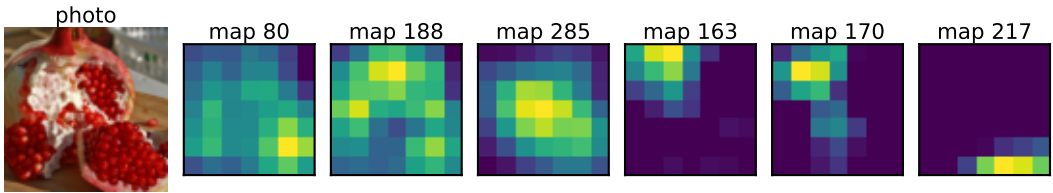
upsampled, overlaid



Grad-CAM = Gradient-weighted Class Activation Mapping (Selvaraju et al., ICCV 2017).

Grad-CAM, step 1: the last feature maps

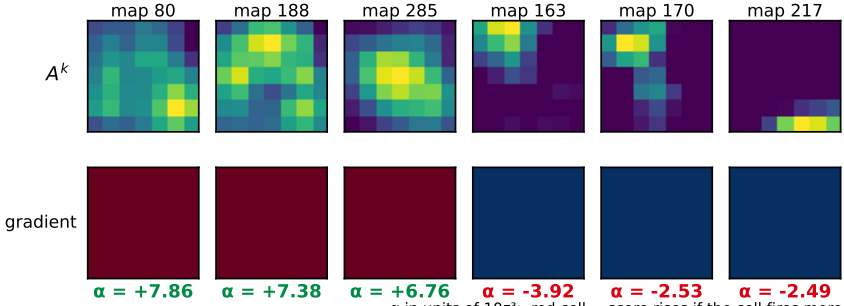
Our photo goes through ResNet-18. Its last conv block outputs **512 feature maps of** 7×7: each cell sits over a 32×32-pixel patch of the photo, and each map lights up where one high-level feature was found. Six of the 512:



Each map says *where*, none says *what for*: some fire on the fruit, some on the background. Next: which maps matter for "pomegranate"?

Grad-CAM, step 2: one weight per map, from the gradient

Ask backprop ([42]): if cell (i, j) of map k fired a little more, how much would the "pomegranate" score y rise? That is $\partial y / \partial A^k_{ij}$. Average it over the 49 cells: one weight per map, $\alpha_k = \frac{1}{49} \sum_{i,j} \partial y / \partial A^k_{ij}$.

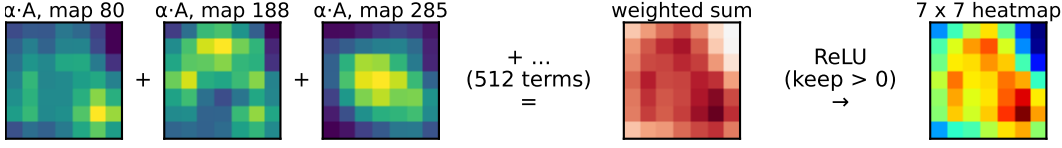


Why is each gradient one flat colour? ResNet-18 ends in a global average pool and one dense layer ([49]), so $y = \sum_k w_k \cdot \text{mean}(A^k)$ and $\partial y / \partial A^k_{ij} = w_k / 49$ in every cell: α_k is just the head's weight for map k. With a deeper head the gradient varies cell by cell; Grad-CAM works either way.

Grad-CAM, step 3: weighted sum, then ReLU

Multiply each map by its weight and add all 512: maps that support "pomegranate" add, maps against it subtract. Then keep only the positive part:

$$L = \mathrm{ReLU}\Big(\sum_{k=1}^{512} \alpha_k A^k\Big) \qquad (7 \times 7)$$

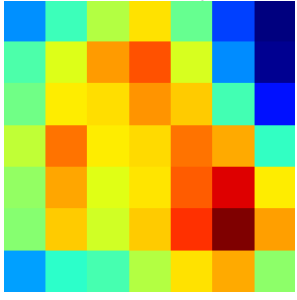


That is how 512 maps become **one** picture: a weighted vote, cell by cell. ReLU drops the evidence *against* the class - we want what argued *for* it.

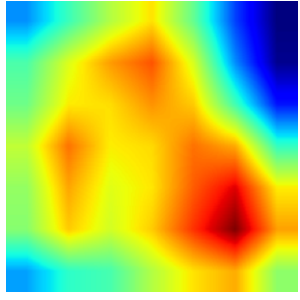
Grad-CAM, step 4: upsample to the photo

The heatmap is 7×7, the photo 224×224. **Upsampling** stretches the grid over the photo: each of the 49 values sits at the centre of its 32×32 patch, and every pixel in between gets a **blend of the four nearest values**, weighted by distance (bilinear interpolation).

7 x 7: one value per cell



bilinear: 224 x 224



laid over the photo



Nothing new is learned here - the 49 numbers are only smoothed so they line up with the pixels. That is why Grad-CAM maps look blurry: their real resolution is 7×7.

The verdict on our photo

No cherry-picking: an ImageNet ResNet-18, one forward and one backward, our photo.

Top-1: **pomegranate**, $p = 0.99$. The evidence sits on the **fruit** - the aril clusters - not on the board or the basket.

Caveat: ImageNet *has* a pomegranate class, so this is the easy case. The audit that matters is on a model trained on *your* data - Gesture Snake runs it on your thumb model.

The interpretability decks ([22]-[24]) asked which **features** matter - importance, SHAP. Grad-CAM asks which **pixels**. Same question, new coordinates.

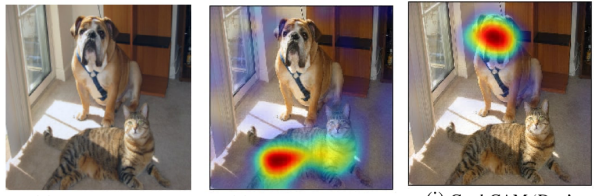
input photo



Grad-CAM, 'pomegranate'



Ask about another class, get another map



(a) Original Image (c) Grad-CAM 'Cat' (i) Grad-CAM 'Dog'

Selvaraju et al., ICCV 2017, Fig. 1 (VGG-16).

The net outputs a score for *every* class; step 2 differentiated the top one. **To ask about another class, backprop its score instead**: `model(x)[0, c].backward()` for class `c`. In our ResNet-18, α_k then comes from row `c` of the head: new weights, new map. Above: "cat" lights up the cat, "dog" the dog.

On the line: on a fruit graded *juice*, ask "why juice?" - the map should sit on the bruise, not on the belt.

What it cannot tell you: *what* - a hot spot on the fruit cannot say colour, shape or sticker. And it sees on a 7×7 grid.

Clever Hans, the vision edition



Berlin, early 1900s: the horse **Clever Hans** "did arithmetic", tapping answers with a hoof. A 1907 investigation showed he was reading his questioner's *unconscious posture*. Right answers, wrong reason.



(a) Husky classified as wolf (b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the "Husky vs Wolf" task.

A wolf-vs-husky classifier, trained where every wolf photo had **snow** in the background. It looked accurate - and was a **snow detector** (Ribeiro et al. 2016 - the LIME paper, [24]).

On the sorting line: is the model grading the **fruit**, or the **conveyor belt** behind it? High accuracy is not right reasons. Grad-CAM is the five-minute check before you trust a model.

Ribeiro et al., "Why Should I Trust You?": Explaining the Predictions of Any Classifier, KDD 2016.

The real budget is labels

GPUs are rented by the hour. Human attention is the expensive part.

Somebody has to draw the boxes

- Your 2,000 labeled photos meant somebody **looked at every single one** - hours of human attention, plus a second pass wherever labelers disagreed.
- Costs climb with detail: a class label is a **click**; a bounding box is a **drawing**; a per-pixel mask is a **tracing**. Each step up costs far more attention per image.
- Real projects spend their budget on **labels and label quality**, not on GPUs.

Pretraining + transfer trade **your labels** for compute **somebody else already spent**. That is why transfer is the default - economically, not just statistically.

Hold that thought: [51]'s detection tasks need those *boxes* - keep the price in mind.

Spending fewer labels

Transfer cut how *many* labels you need. Four habits cut what each one costs:

- **Try zero labels first.** CLIP's zero-shot trick: "a photo of a bruised pomegranate" vs "a photo of a perfect pomegranate". Good enough? Done. If not, you learn *where* it is confused.
- **Let a model draft, let people correct.** Accepting or fixing a pre-filled label is quicker than labeling from nothing - for boxes and masks even more ([51]).
- **Label what the model is unsure about**, not a random sample: a confident, correct prediction teaches it little (*active learning*).
- **Consistency beats volume.** Two labelers who disagree on "market or juice" teach the model noise. Write the grading rule down; check agreement on a shared sample.

The cheapest label is the one you never had to make - and the next cheapest is one a model drafted for you.

Recap

- With 2,000 photos, **never start from scratch** - reuse the feature hierarchy someone already paid to learn.
- A pretrained net = **trunk** (photo $\rightarrow$ 512 numbers) + **head** (512 numbers $\rightarrow$ classes). Transfer keeps the trunk and swaps the head.
- **Feature extraction**: train the head only (1,539 numbers). **Fine-tune**: head first, then unfreeze (LP-FT) at a small LR - and "just the top block" is 75% of ResNet-18.
- Frozen trunk $\Rightarrow$ **BN to eval mode**; feed the net **its** preprocessing.
- **Grad-CAM**: per-class maps of *where* (not *what*) - the five-minute Clever Hans check.
- **Labels are the budget.** Transfer, zero-shot, model drafts and active learning all spend fewer of them.

Next: the classifier is built and audited. But the belt camera sees *ten* fruit at once - object detection and segmentation ([51]).